



FIXTRACK

Customer + Technician Dashboard User Manual

How to use the service workspace,
and how to clone and run the Laravel system.

For customers, technicians, developers, and maintainers

Edition

August 2026

Platform

Laravel + Livewire

01 | ORIENTATION

Start here

This guide explains the current FixTrack customer and technician workspaces. It also documents the safe setup sequence after cloning the Laravel repository from GitHub.

Live-data rule

The dashboards are role-scoped. Customers see their own bookings, payments, reviews, queue entries, and support tickets. Technicians see available requests plus only their assigned jobs, earnings, ratings, and verification profile.

Role routing

Role	Entry URL	What the role can see	Primary outcome
Customer	/customer	Own service records and account activity	Request and track home services
Technician	/technician	Available requests and own assignments	Accept, perform, and close jobs
Admin	/admin/dashboard	Platform-wide operational records	Manage dispatch, quality, finance, and access

The normal browser path



After a successful login, /dashboard routes the user by role. Public registration creates customer or technician accounts. Admin accounts are provisioned separately and are not offered as a public registration option.

Customer: <http://127.0.0.1:8000/customer>
Technician: <http://127.0.0.1:8000/technician>
Admin: <http://127.0.0.1:8000/admin/dashboard>

02 | CUSTOMER DASHBOARD

Customer workspace

The Customer dashboard is the service requester's home base. It combines current work, booking history, service payments, queue status, feedback, support, and account settings.

Navigation map

Area	Modules	Use it for
Home	Dashboard	See ongoing bookings, completed jobs, total bookings, open support, next visit, and recent bookings.
Bookings	Book a Service; My Bookings; Walk-in Queue	Create a request, filter history, open details, or join a live queue.
Service journey	Quotations; Payments	Review assessment status and payment records tied to your bookings.
Account	Addresses; Ratings & Reviews; Notifications; Support & Help; Settings	Manage locations, feedback, updates, support requests, profile, security, and appearance.

Dashboard cards are live records

- Ongoing booking counts your own pending, matching, assigned, en route, and in-progress bookings.
- Completed jobs counts your own completed bookings.
- Total bookings counts all bookings owned by the authenticated customer.
- Open support counts your own open and in-progress support tickets.

Empty states are meaningful

A message such as 'No active service' or 'Your booking history will appear here' means the scoped database query found no matching record. It is not a demo fallback.

Refresh behavior

Dashboard, My Bookings, Walk-in Queue, and Payments use lightweight live-update checks. Other customer sections refresh on navigation, manual refresh, or after an action such as booking, reviewing, or sending support.

03 | CUSTOMER WORKFLOW

Book and track a service

Use this flow when you need a home service. The form validates the request before it is written to the bookings table.

1

Open Book a Service

Choose Quick booking to start matching now, or Schedule a visit to select a future date and time.

2

Enter service details

Choose an active service type, confirm your name, mobile number, service address, and optional description. Scheduled bookings require a future date and time.

3

Submit the request

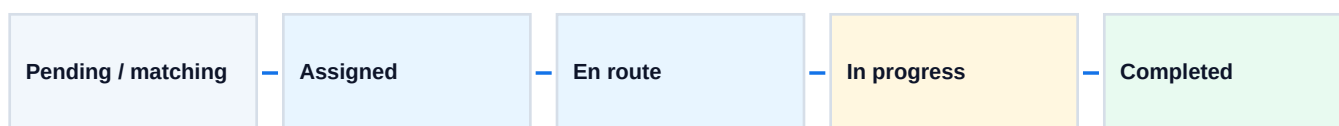
FixTrack creates a real booking reference, links it to your customer account, and places it in matching or pending status according to platform settings.

4

Track My Bookings

Use search and status filters, then open a row to see the address, schedule, booking type, description, and current status.

Typical service status flow



A booking may also be cancelled. The visible status always comes from the booking record, so the customer view and technician view stay aligned.

Walk-in Queue

Open Walk-in Queue, choose a service, enter your phone number, add optional notes, and submit. Your queue entry receives a queue number and is scoped to your account. The page reports waiting, being served, and completed counts from walk_in_entries.

Before you submit

Check the service address and phone number. These values are stored with the booking or queue entry and are used by dispatch and the assigned technician.

04 | CUSTOMER OPERATIONS

Payments, reviews, and support

Payments

Payments are connected to your bookings. The page shows Total paid, Paid records, and Pending records. Paid totals include completed bookings with payment status paid. Open a row to confirm the booking reference, amount, method, paid date, and status.

Ratings & Reviews

Completed bookings that do not yet have a review appear in the Leave feedback area. Choose a rating from 1 to 5, add an optional comment, and submit once. Your average rating and submitted review count are calculated from your own review records.

Quotations

The current UI reserves this area for technician assessments and quotations. This version does not yet persist quotation records, so the page is informational until that data model is introduced.

Notifications

Notifications combine your recent booking updates and support ticket updates. They are derived from the same relational records shown in the dashboard and support pages.

Support & Help

1

Create a request

Enter a subject, choose Booking, Payment, Technical, Account, or Other, set Low, Normal, or High priority, and write the message.

2

Track the ticket

Your support table shows the reference, subject, category, priority, updated time, and current status.

Account settings

Settings links to Profile, Security, and Appearance. Keep your name and email accurate because relational records use the authenticated account identity.

05 | TECHNICIAN DASHBOARD

Technician workspace

The Technician dashboard is an operational workspace. It is designed around availability, incoming requests, assigned work, route progress, earnings, quality, verification, and support.

Navigation map

Area	Modules	Use it for
Workspace	Dashboard; Job Requests; My Jobs; Schedule	See the next action, review available work, manage assigned jobs, and filter appointments.
Operations	Dispatch & Routes	Follow active route assignments, current job, location signal, and availability.
Finance and quality	Earnings; Ratings & Reviews	Review completed-job payments, totals, payout status, rating average, and feedback.
Account	Verification & Profile; Notifications; Support & Help	Monitor approval status, service categories, updates, and support tickets.

Dashboard cards

- Today's jobs counts your assigned bookings scheduled today or created today.
- Active jobs counts your assigned bookings in assigned, en route, or in-progress status.
- Completed jobs counts your assigned completed bookings.
- Average rating is calculated from your published reviews.

Availability matters

Use Go online when you are ready for requests. Accepting a request also requires an active technician account and approved verification.

06 | TECHNICIAN WORKFLOW

From request to completed job

This is the core technician flow. Each action updates the booking record and is visible to the appropriate customer and admin views.

1

Go online

From Dashboard or Job Requests, switch availability to available. Go offline when you are not accepting new work.

2

Review Job Requests

Available requests are unassigned bookings in pending or matching status. Search by reference, customer, service, or address.

3

Accept or decline

Accept moves the booking to assigned and adds it to My Jobs. Decline removes it from your request queue without assigning it to you.

4

Start the route

For an assigned job, choose Start route. The booking moves to en_route and appears in Dispatch & Routes.

5

Start the repair

Choose Start repair after the route begins. The booking moves to in_progress.

6

Complete the job

Choose Complete job after the repair is done. The booking moves to completed and a pending payment record is created when payments are available.

Job status flow

Pending / matching

Assigned

En route

In progress

Completed

Confirmation is intentional

Every accept, decline, route, repair, and completion action uses a confirmation step. If an action fails, the dashboard reports the server-side error instead of silently changing the row.

07 | TECHNICIAN OPERATIONS

Schedule, routes, earnings, and quality

Schedule

Use Upcoming, Today, Completed, or All appointments to narrow your assigned bookings. Search by reference, customer, service, or address. The summary cards show today, upcoming, in-progress, and completed counts.

Dispatch & Routes

The route view shows active assigned jobs, the current job, location signal, latitude and longitude when available, last seen time, and your current availability. Keep location sharing enabled while available if dispatch needs live coordination.

Earnings

Earnings are calculated from payment rows for your completed assigned bookings. Total earned and Earned today sum paid amounts. Paid jobs and Pending payments count records by payment status.

Ratings & Reviews

The page shows your published reviews, average rating, and five-star review count. Reviews are linked to the technician_id on the review record.

Verification & Profile

This page reads your technician verification, service categories, years of experience, service area, risk level, decision reason, and attached documents. A new technician registration creates a submitted verification record; admin approval is required before accepting requests.

Notifications and Support

Notifications summarize available requests, active assignments, and verification status. Support shows tickets where you are the requester or assigned staff member.

Refresh expectations

Job Requests polls about every 5 seconds. Dashboard, My Jobs, and Dispatch & Routes poll about every 10 seconds. Earnings and Verification refresh about every 60 seconds. Schedule, ratings, notifications, and support refresh on navigation or manual refresh.

08 | DEVELOPER SETUP

Clone the Laravel system

GitHub stores the application source, migrations, configuration templates, and frontend source. It does not store local secrets, installed dependencies, runtime caches, compiled assets, or database records.

Prerequisites

- Git
- PHP 8.3 or newer
- Composer
- Node.js and npm
- MySQL or another configured Laravel database connection
- A terminal with permission to write storage and bootstrap/cache

Clone and enter the project

```
git clone <YOUR_GITHUB_REPOSITORY_URL> fixtrack-final
cd fixtrack-final
```

Why files appear to be missing after cloning

Not in GitHub	Why	Restore it with
.env	Local secrets and database settings are ignored.	cp .env.example .env, then edit it.
vendor/	Composer dependencies are generated locally.	composer install
node_modules/	npm dependencies are generated locally.	npm install
public/build/	Vite output is generated, not source.	npm run build or npm run dev
storage runtime files	Logs, cache, sessions, and uploads are environment data.	Create writable folders and run storage:link.
Database rows	Git does not carry MySQL data.	Run migrations and restore a database backup if needed.

09 | DEVELOPER SETUP

Configure the environment

Create the local environment file

```
cp .env.example .env
composer install
php artisan key:generate
```

Keep .env private. Do not commit it or paste database passwords, APP_KEY values, mail credentials, or API secrets into GitHub.

Configure MySQL

```
CREATE DATABASE fixtrack CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=fixtrack
DB_USERNAME=your_mysql_user
DB_PASSWORD=your_mysql_password
```

The SQL statement is run in MySQL, not pasted into .env. Put the DB_* values in .env, then run migrations from the project directory.

Install schema and runtime links

```
php artisan migrate --force
php artisan storage:link
npm install
npm run build
```

Existing database warning

Use php artisan migrate for an existing environment. Do not use migrate:fresh or db:wipe unless you intentionally want to delete every table and record. The legacy demo cleanup migration runs as part of the normal migration sequence.

10 | DEVELOPER SETUP

First-run checklist

This repository intentionally does not seed operational demo bookings, payments, reviews, or queue records. A fresh database therefore needs real setup data before customers and technicians can use the full workflow.

1

Provision the first admin

Public registration accepts customer and technician roles only. Create a controlled superadmin/admin account through your deployment provisioning process, then log in at `/admin/dashboard`.

2

Initialize platform settings

Open Platform Settings and initialize the live controls. These settings govern booking matching, queue availability, dispatch limits, notifications, finance, and quality policies.

3

Create service catalog entries

Open Service Catalog and add active service offerings with a code, name, category, base price, and description. Customers can only choose active catalog services.

4

Create real user accounts

Use `/register` to create a customer or technician account. Technician registration collects phone, category, service area, and years of experience, then starts verification in submitted status.

5

Approve technicians

An admin reviews Technician Verification. A technician needs approved verification and an active account before accepting job requests.

6

Run a real end-to-end test

Customer creates a booking, technician accepts it, moves it through `en_route` and `in_progress`, completes it, and customer confirms payment/review behavior.

Run the local stack

```
composer run dev
# starts Laravel server, queue listener, logs, and Vite
```

If you prefer separate terminals, run `php artisan serve` in one terminal and `npm run dev` in another. The default web URL is usually `http://127.0.0.1:8000`.

11 | OPERATIONS

Verify and troubleshoot

Quick health checks

```
php artisan about
php artisan route:list --path=customer
php artisan route:list --path=technician
php artisan migrate:status
curl http://127.0.0.1:8000/up
```

Symptom	Likely cause	First check
APP_KEY missing	The cloned project has no .env.	Copy .env.example, then run php artisan key:generate.
SQLSTATE connection error	MySQL is stopped or DB_* values are wrong.	Check MySQL, database name, username, password, host, and port.
Vite manifest error	Frontend assets were not built.	Run npm install and npm run build, or keep npm run dev running.
Permission denied in storage	Runtime folders are not writable.	Check storage and bootstrap/cache permissions; run storage:link.
No services in Book a Service	Service Catalog is empty or inactive.	Sign in as admin and create active service catalog entries.
Technician cannot accept	Verification is not approved or account is inactive.	Review Verification & Profile and admin Technician Verification.
No rows in a table	There are no matching live records for this role.	Confirm the authenticated account and create a real workflow record.

Before committing to GitHub

- Never commit .env, database passwords, APP_KEY, uploaded documents, logs, or local database exports.
- Keep composer.lock and package-lock.json committed so installs are reproducible.
- Run vendor/bin/pint --dirty --format agent for changed PHP files and php artisan test --compact.
- Run npm run build when frontend source changes.
- Back up MySQL data separately from the Git repository.

Deployment principle

GitHub restores code. Composer, npm, .env, migrations, storage, and database backup restore the runnable system.